

Sandbox Project Style Guide

This is a reusable interface style guide for a compact technical web application UI.

Study the HTML/CSS examples below and use them as the visual reference for the new project. Treat the example as a design system, not as a fixed application. Your goal is to apply the same interface language to the project: layout, typography, colours, buttons, accordion menus, sliders, checkboxes, list items, panels, icons, and interaction states.

Use this guide as the source of truth for the visual style. Do not copy the exact project content unless it is useful as a generic placeholder. Instead, recreate the reusable patterns in clean HTML, CSS, and JavaScript.

The finished UI should feel compact, technical, precise, and professional. It should look like a functional creative/research tool, not a marketing website or dashboard template.

The sandbox logo below will be attached:



Cover the following areas:

1. Overall Interface Style

Explain the intended visual direction:

- Compact technical interface

- Dark professional tool palette
- Precise, functional layout
- Suitable for creative tools, viewers, editors, research interfaces, and control panels
- Avoid marketing-style cards, large hero sections, decorative gradients, oversized rounded UI, or dashboard-style layouts

2. Colour Palette

Describe the palette and how each colour should be used:

- Dark charcoal / black backgrounds
- Slightly lighter panel surfaces
- Mid-grey item strips
- Low-contrast borders
- Off-white primary text
- Muted grey secondary text
- Bright blue accent colour for active states, highlights, focus, and selected controls
- Red for destructive actions such as delete

Include a CSS example using variables:

```
```css
:root {

 --bg: #101014;

 --topbar: #1d1d22;

 --panel: #1a1a20;

 --surface: #26272f;

 --surface-mid: #2c2d35;

 --border: #343642;

 --text: #f2f3f7;

 --muted: #a5abc0;

 --subtle: #8f96a8;

 --accent: #3b97ee;

 --accent-fill: #163452;

 --danger: #ff3838;

 --mono: "IBM Plex Mono", "Cascadia Mono", Consolas, monospace;

 --sans: Inter, system-ui, sans-serif;

}
```
```

3. Typography

Explain the text hierarchy:

- Main title: larger, clean, confident
- Subtitle: smaller, muted, descriptive
- Accordion headings: uppercase monospace, slightly larger than small labels
- Labels: consistent size across controls
- Notes and placeholder text: mid-grey, readable but secondary
- Use letter spacing for technical labels

Include CSS examples:

```
```css
.app-title {
 font-family: var(--sans);
 font-size: 26px;
 font-weight: 700;
 color: var(--text);
}

.app-subtitle {
 font-size: 15px;
 color: var(--muted);
}

.section-title,
.control-label {
 font-family: var(--mono);
 font-size: 11px;
 font-weight: 700;
 text-transform: uppercase;
 letter-spacing: 0.08em;
}
```
```

4. Header And Branding

Explain how the top header should work:

- Fixed-height dark header
- Logo/icon at the left

- Product or project name beside the icon
- Optional actions aligned to the right
- Icon should be an attached image asset, not a drawn placeholder

Include HTML and CSS examples:

```
```html
<header class="topbar">

 <div class="brand">

 <div>

 <h1>Project Interface</h1>

 <p>Compact technical UI pattern library</p>

 </div>

 </div>

 <div class="top-actions">

 <button class="button">Choose File</button>

 <button class="button primary">Preview</button>

 </div>

</header>
```css
.topbar {
  height: 48px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: var(--topbar);
  border-bottom: 1px solid var(--border);
  padding: 0 16px;
}

.brand {
  display: flex;
```

```
    align-items: center;

    gap: 10px;
}

.brand-icon {

    width: 28px;

    height: 28px;

    object-fit: contain;
}

...

```

5. Buttons

Explain button behaviour:

- Compact rectangular buttons
- Thin borders
- Dark default state
- Blue active state
- White text on hover
- Delete/destructive buttons turn red with white text on hover
- Use icons where helpful, but keep icon style consistent

Include examples:

```
```html

<button class="button">Default</button>

<button class="button primary">Primary</button>

<button class="button active">Active</button>

<button class="button danger">Delete</button>

...

```

```
```css

.button {

    background: var(--surface);

    border: 1px solid var(--border);

    color: var(--muted);

    border-radius: 4px;

    padding: 8px 10px;
}

```

```
font-family: var(--mono);
font-size: 11px;
font-weight: 700;
text-transform: uppercase;
cursor: pointer;
}
.button:hover {
border-color: var(--accent);
color: #fff;
}
.button.active,
.button.primary {
background: var(--accent-fill);
border-color: var(--accent);
color: var(--accent);
}
.button.active:hover,
.button.primary:hover {
color: #fff;
}
.button.danger:hover {
background: var(--danger);
border-color: var(--danger);
color: #fff;
}
...

```

6. Accordion Menus

Explain accordion behaviour:

- Used for sidebar sections
- Multiple sections may be open at once
- Heading bar is compact and technical
- Plus/minus or chevron indicator on the right

- Body contains compact controls arranged in rows or grids

Include HTML, CSS, and JavaScript examples:

```
```html
<section class="accordion open">

 <button class="accordion-heading" type="button">

 Camera Data

 –

 </button>

 <div class="accordion-body">

 <div class="control-row">

 <label>

 Position X

 <input type="number" value="0">

 </label>

 <label>

 Position Y

 <input type="number" value="0">

 </label>

 <label>

 Position Z

 <input type="number" value="0">

 </label>

 </div>

 </div>

</section>
```

```css
.accordion {

 border-bottom: 1px solid var(--border);

}
```

```
.accordion-heading {
 width: 100%;
 display: flex;
 justify-content: space-between;
 align-items: center;
 background: var(--surface-mid);
 color: var(--text);
 border: 0;
 padding: 10px 12px;
 font-family: var(--mono);
 font-size: 11px;
 font-weight: 700;
 text-transform: uppercase;
 letter-spacing: 0.08em;
 cursor: pointer;
}

.accordion-heading:hover {
 color: #fff;
}

.accordion-body {
 display: none;
 padding: 10px;
 gap: 10px;
}

.accordion.open .accordion-body {
 display: grid;
}

...

```js
document.querySelectorAll(".accordion-heading").forEach((heading) => {
```



```

heading.addEventListener("click", () => {

  const accordion = heading.closest(".accordion");

  accordion.classList.toggle("open");

  const indicator = heading.querySelector(".accordion-indicator");

  indicator.textContent = accordion.classList.contains("open") ? "-" : "+";

});

});
...

```

7. Sidebar / Control Panel

Explain the menu panel:

- Fixed-width vertical control panel
- Full height
- Scrollable
- Dark background
- Accordion menus stacked vertically
- Compact spacing
- No oversized cards

Include layout CSS:

```

```css

.layout {

 display: grid;

 grid-template-columns: 260px 1fr;

 min-height: 100vh;

 background: var(--bg);

}

.sidebar {

 background: var(--panel);

 border-right: 1px solid var(--border);

 overflow-y: auto;

}

.workspace {

 min-width: 0;

 background: #08090b;

```

```
}
...

```

## 8. Inputs And Form Controls

Explain input styling:

- Number fields, text fields, dropdowns, and textareas should share a common style
- Compact height
- Dark background
- Thin border
- Blue focus state
- Textareas can be taller for notes or annotations

Include examples:

```
```html  
  
<label>  
  
  <span class="control-label">Threshold</span>  
  
  <input type="number" value="0.6">  
  
</label>  
  
<textarea placeholder="Annotation or note text goes here."></textarea>  
...  

```

```
```css  

input,
select,
textarea {
 width: 100%;
 background: #14151a;
 border: 1px solid var(--border);
 color: var(--text);
 border-radius: 4px;
 padding: 7px 8px;
 font-family: var(--mono);
 font-size: 12px;
}

textarea {
 min-height: 96px;
}
```

```
 resize: vertical;

 color: var(--subtle);
}

input:focus,
select:focus,
textarea:focus {
 outline: none;
 border-color: var(--accent);
}
...

```

## 9. Sliders

Explain slider usage:

- Used for continuous values such as speed, size, brightness, threshold
- Label and value should not overlap
- Slider should leave space for numeric value
- Accent colour should appear on the thumb and active focus states

Include example:

```
```html
<div class="slider-control">
  <div class="slider-header">
    <span class="control-label">Speed</span>
    <span class="slider-value">0.50</span>
  </div>
  <input type="range" min="0" max="1" step="0.01" value="0.5">
</div>
...
```css
.slider-header {
 display: flex;
 justify-content: space-between;
 gap: 12px;
}

```

```
input[type="range"] {
 accent-color: var(--accent);
}
...
```

## 10. Checkboxes

Explain checkbox styling:

- Compact labels
- Same label size as other controls
- Accent colour when checked
- Keep checkbox rows aligned and predictable

Include example:

```
```html  
<label class="check-row">  
  <input type="checkbox" checked>  
  <span>Show</span>  
</label>  
...
```

```
```css  
.check-row {
 display: flex;
 align-items: center;
 gap: 8px;
 font-family: var(--mono);
 font-size: 11px;
 font-weight: 700;
 color: var(--text);
}
.check-row input {
 width: 14px;
 height: 14px;
 accent-color: var(--accent);
}
```

...

## 11. Readouts And Data Displays

Explain readout styling:

- Use three-column rows for X/Y/Z values
- Use uppercase axis labels
- Keep numbers aligned and easy to scan
- Use monospace font

Include example:

```
```html
<div class="readout-grid">
  <div><span>X</span><strong>-2.42</strong></div>
  <div><span>Y</span><strong>-0.08</strong></div>
  <div><span>Z</span><strong>-0.17</strong></div>
</div>
```
```

```
```css
.readout-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 6px;
}
.readout-grid div {
  background: #14151a;
  border: 1px solid var(--border);
  border-radius: 4px;
  padding: 6px;
  font-family: var(--mono);
}
.readout-grid span {
  display: block;
  font-size: 11px;
  color: var(--muted);
}
```

```

}

.readout-grid strong {

  font-size: 12px;

  color: var(--text);

}
...

```

12. List Items

Explain list item styling:

- Used for bookmarks, hotspots, nodes, layers, saved views, or annotations
- Top row can use a mid-grey strip
- Text should be slightly larger than small labels
- Icons should remain readable
- Thumbnails or previews should be full width when useful

Include example:

```

```html

<article class="list-item">

 <div class="item-heading">

 <input value="Editable Item Title">

 <button class="icon-button">✕</button>

 </div>

 <div class="preview">Thumbnail / Preview Area</div>

 <div class="item-actions">

 <button class="icon-button">↗</button>

 <button class="icon-button">🔄</button>

 <button class="icon-button danger">🔥</button>

 </div>

 <textarea>Annotation or note text goes here.</textarea>

</article>
...

```css

.list-item {

```

```
background: #15161b;

border: 1px solid var(--border);

border-radius: 4px;

padding: 8px;

display: grid;

gap: 8px;
}

.item-heading {

background: var(--surface-mid);

border-radius: 4px;

padding: 6px;

display: flex;

gap: 6px;
}

.preview {

min-height: 120px;

border: 1px solid var(--border);

display: grid;

place-items: center;

color: var(--subtle);

font-family: var(--mono);

font-size: 13px;
}

.item-actions {

display: grid;

grid-template-columns: repeat(4, 1fr);

gap: 6px;
}

...

```

13. Icons

Explain icon usage:

- Use icons for repeated compact actions
- Keep icons consistent in size, stroke, and colour
- Do not replace existing functional icons unless requested
- Pair icons with tooltips when the action is not obvious
- Use image logo assets for brand marks

14. Interaction Rules

Explain expected interactions:

- Hover states should be visible but not noisy
- Active states should be clear
- Destructive actions should be visually distinct
- Accordion headings should be clickable across the full width
- Focus states must be visible for keyboard use
- UI should remain compact and usable on smaller screens

15. Reusable Starter Template

Provide a minimal complete HTML file showing:

- Header with icon
- Sidebar with accordion menus
- Buttons
- Inputs
- Sliders
- Checkboxes
- List item
- Basic accordion JavaScript

Use separate code blocks for HTML, CSS, and JavaScript.

Keep the explanation concise, practical, and developer-friendly. Avoid describing this as a marketing layout. Present it as a reusable interface system for technical creative tools.